UNIT – 2 LAB EXERCISE

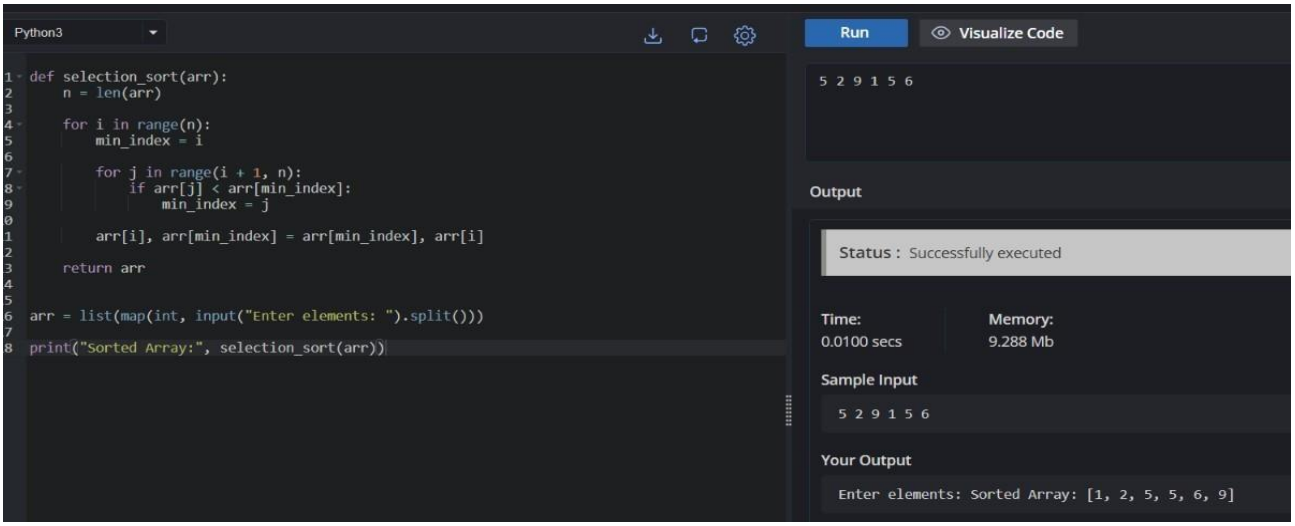
Name: Jeeva
Reg no: 192421334

1. Write a program to perform the following
    - An empty list
    - A list with one element
    - A list with all identical elements
    - A list with negative numbers

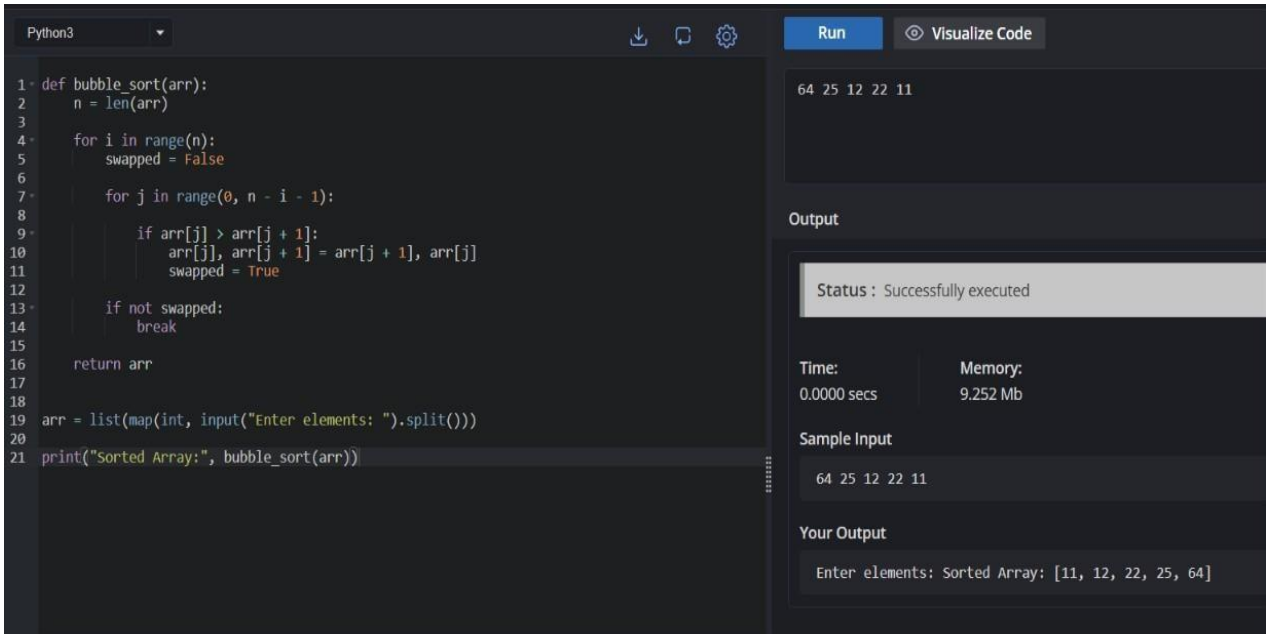
```python
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


# Test Cases
test_cases = [
    [],
    [1],
    [7, 7, 7, 7],
    [-5, -1, -3, -2, -4]
]

for arr in test_cases:
    print("Input :", arr)
    print("Output:", selection_sort(arr.copy()))
    print()
```

```
Enter Input here

If your code takes input, add it in the above box before runnir

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.844 Mb

Your Output

Input : []
Output: []

Input : [1]
Output: [1]

Input : [7, 7, 7, 7]
Output: [7, 7, 7, 7]
```

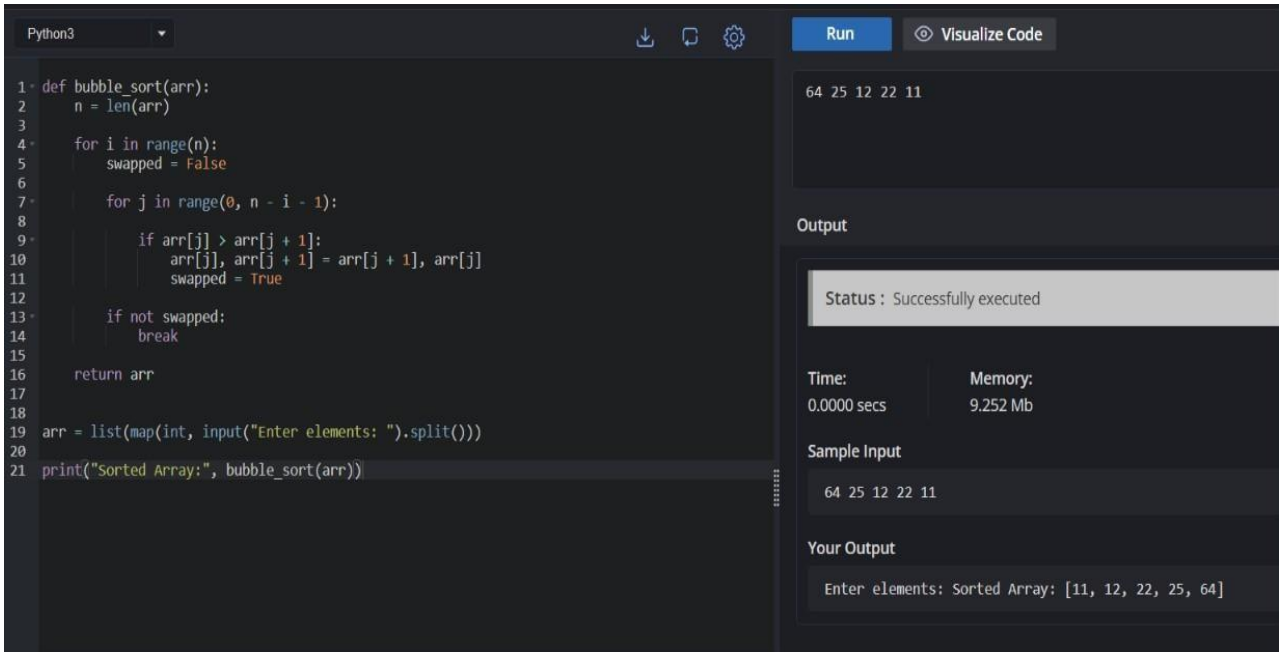
2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.



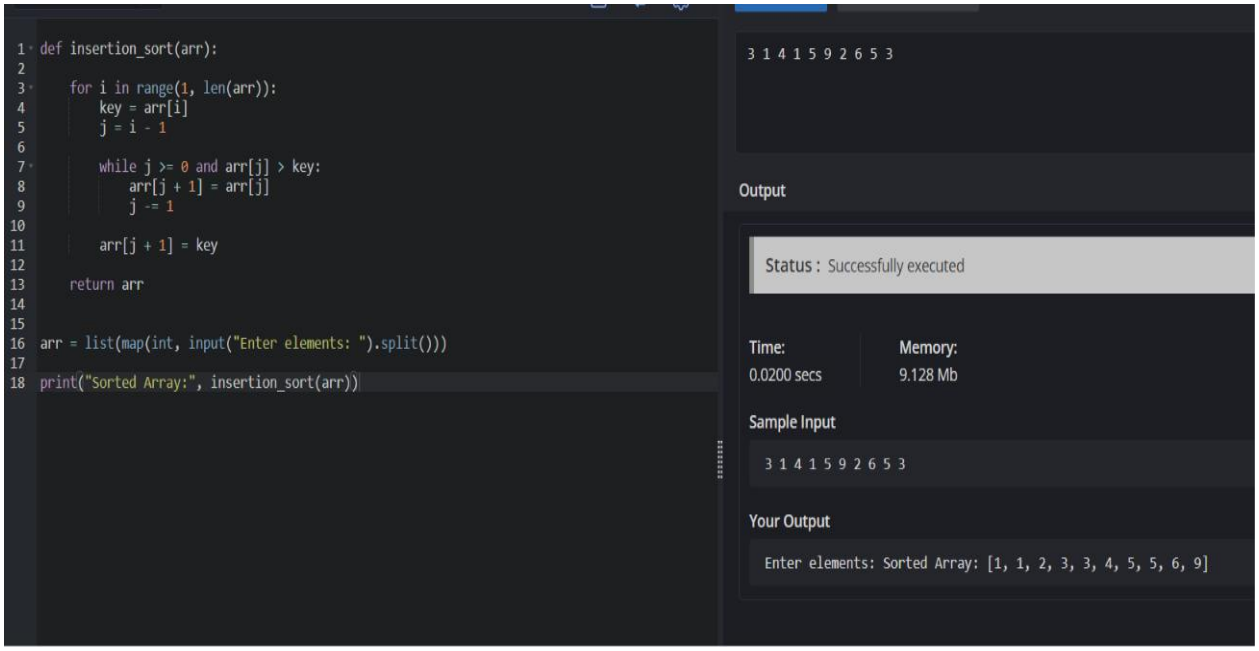
3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.



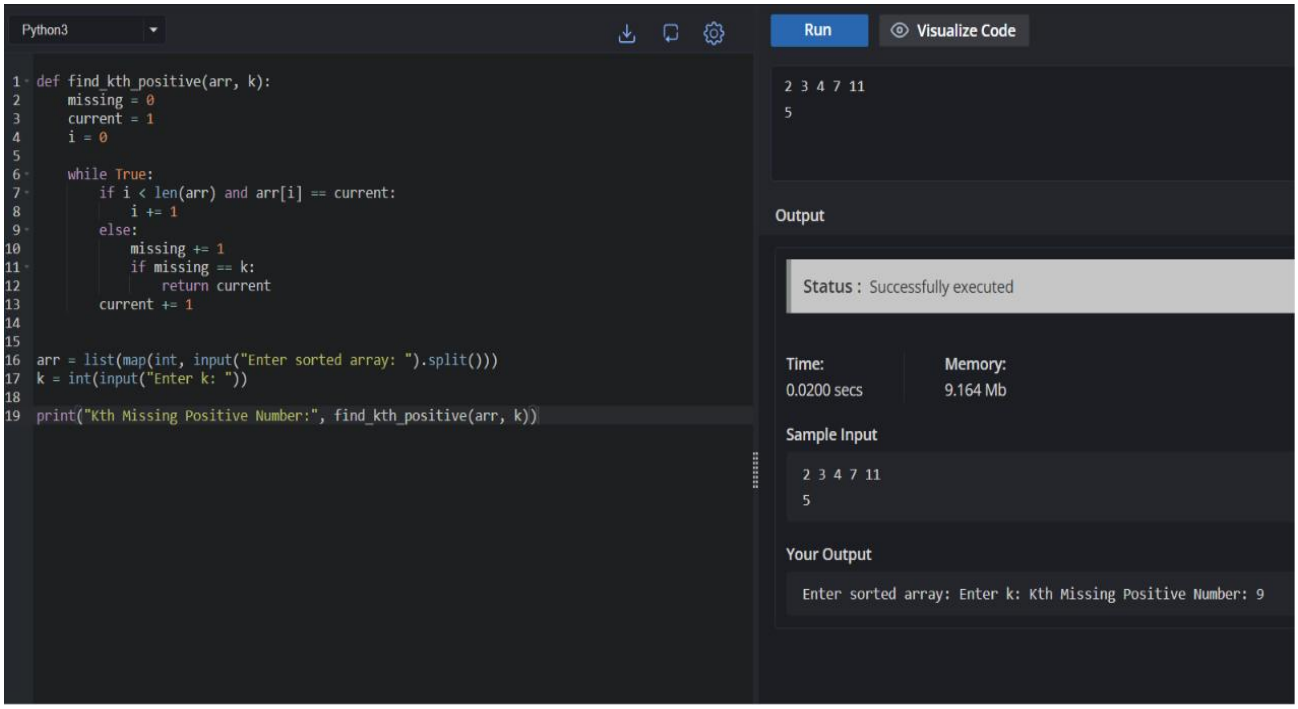
4. Test your optimized function with the following lists:
    - Input: [64, 25, 12, 22, 11]
    - Expected Output: [11, 12, 22, 25, 64]
    - Input: [29, 10, 14, 37, 13]
    - Expected Output: [10, 13, 14, 29, 37]
    - Input: [3, 5, 2, 1, 4]
    - Expected Output: [1, 2, 3, 4, 5]



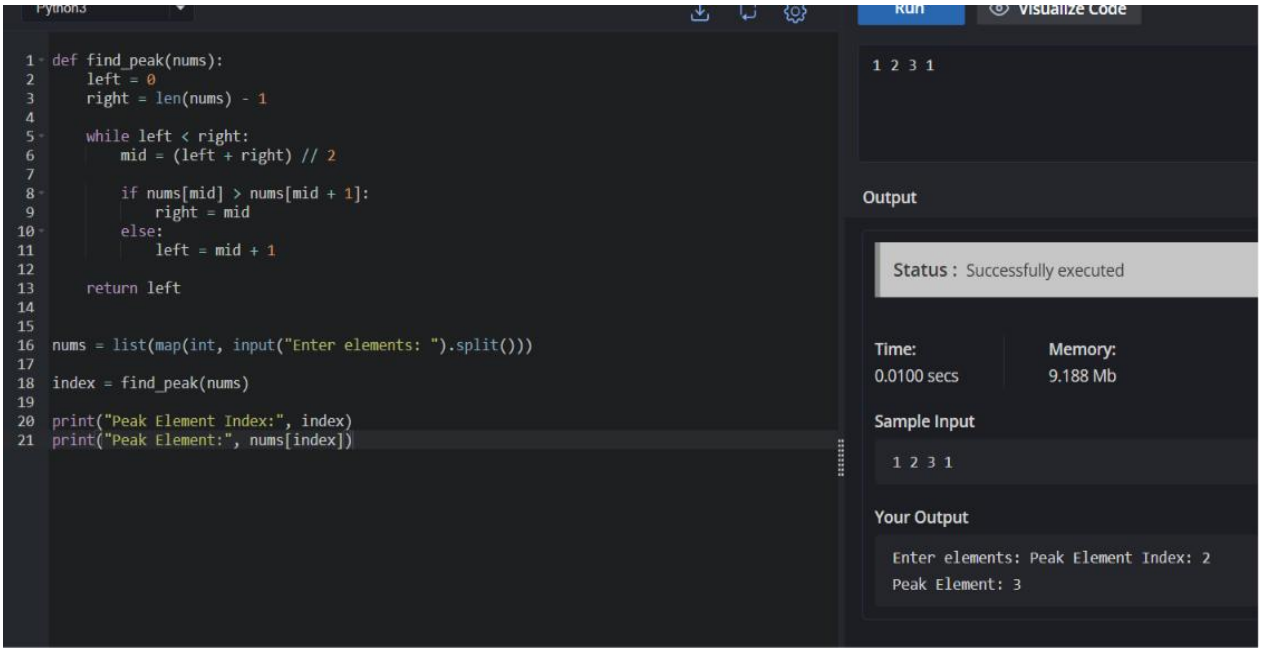
5. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome



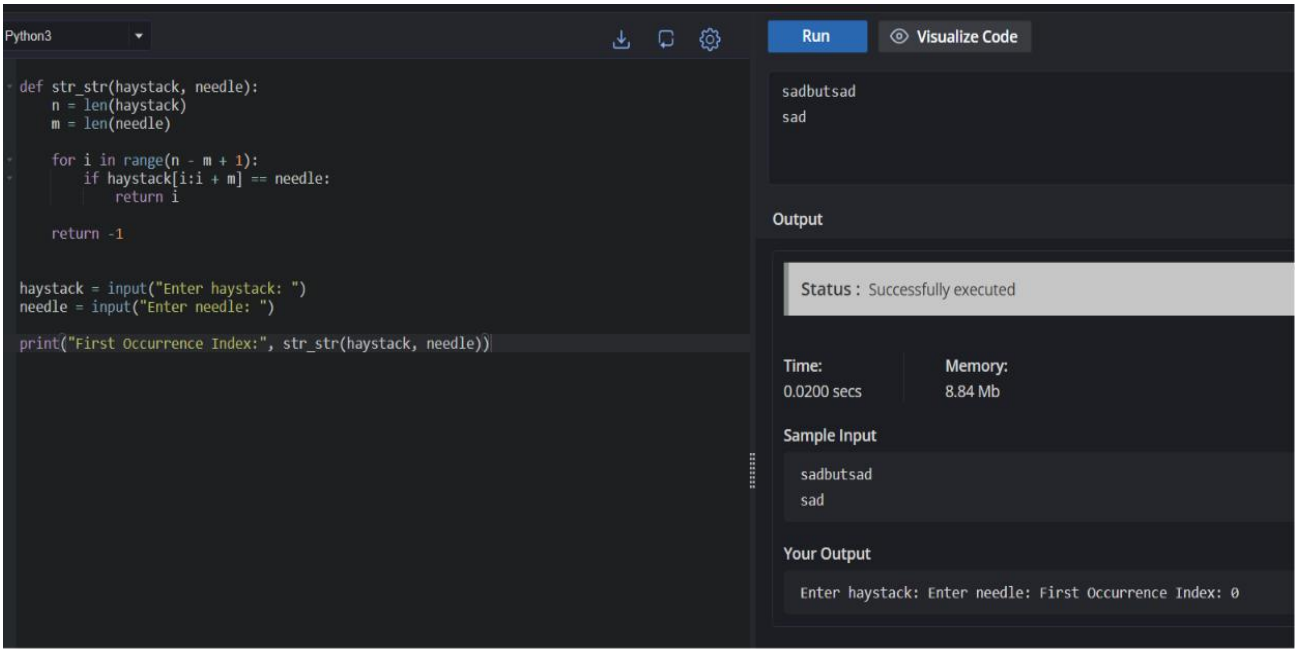
6. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return the kth positive integer that is missing from this array.



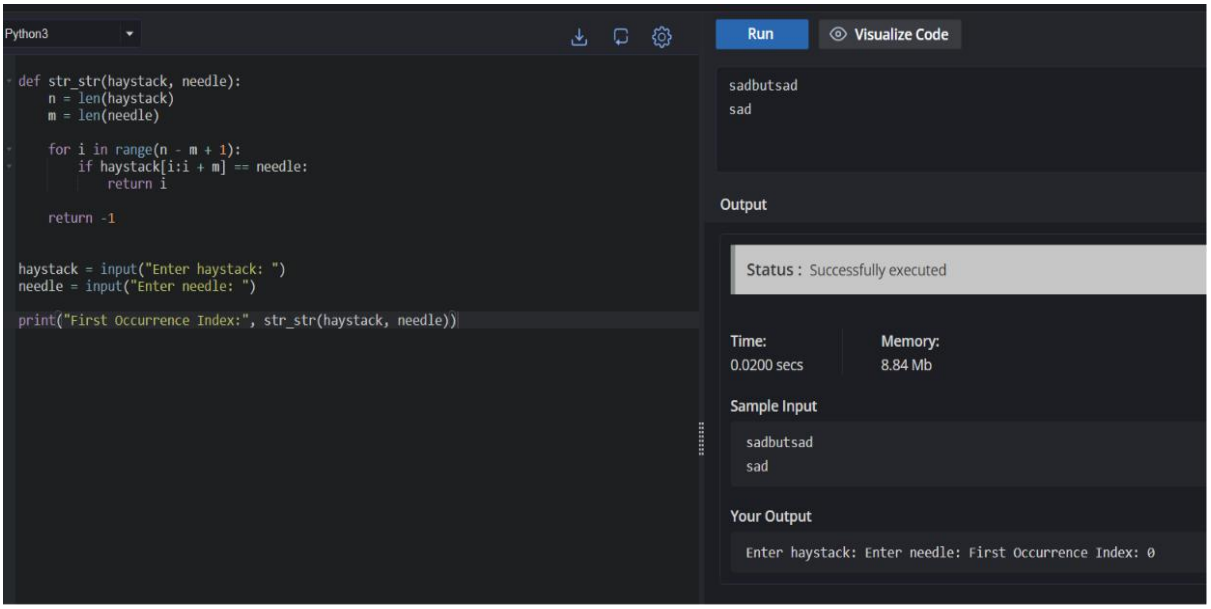
7. A peak element is an element that is strictly greater than its neighbors. Given a 0indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in O(log n) time.



8. Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.



9. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string



10. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

```python
import math

def closest_pair(points):
    min_dist = float('inf')
    pair = ()

    n = len(points)

    for i in range(n):
        for j in range(i + 1, n):

            dist = math.sqrt((points[i][0] - points[j][0]) ** 2 +
                             (points[i][1] - points[j][1]) ** 2)

            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])

    return pair, min_dist


points = [(1, 2), (4, 5), (7, 8), (3, 1)]

pair, distance = closest_pair(points)

print("Closest Pair:", pair)
print("Minimum Distance:", distance)
```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0100 secs

Memory: 9.244 Mb

Your Output

```
Closest Pair: ((1, 2), (3, 1))
Minimum Distance: 2.23606797749979
```

11. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

```python
import math

def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)


def closest_pair(points):
    n = len(points)
    min_dist = float('inf')
    pair = None

    for i in range(n):
        for j in range(i + 1, n):
            d = distance(points[i], points[j])

            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])

    return pair, min_dist


n = int(input("Enter number of points: "))
points = []

for i in range(n):
    x, y = map(int, input(f"Enter point {i+1} (x y): ").split())
    points.append((x, y))

pair, dist = closest_pair(points)

print("\nClosest Pair:", pair)
print("Minimum Distance:", dist)
print("Time Complexity: O(n^2)")
```

```
1 4
2 7
9 5
3 6
```

Output

Status : Successfully executed

Time: 0.0100 secs

Memory: 9.42 Mb

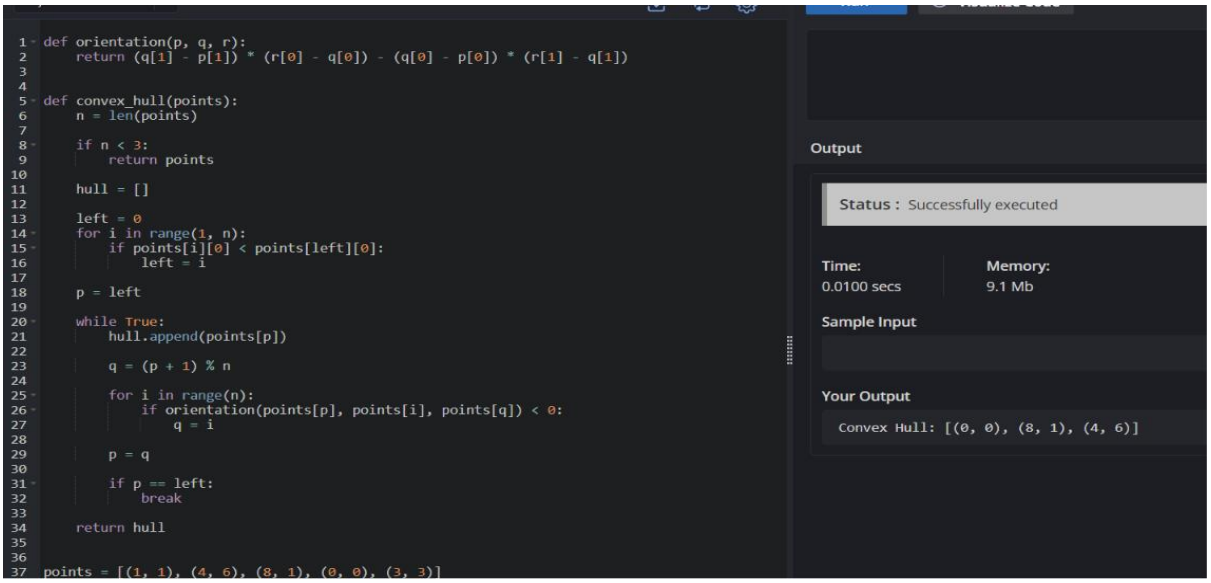
Sample Input

```
4
1 4
2 7
9 5
3 6
```

Your Output

```
Enter number of points: Enter point 1 (x y): Enter point 2 (x y): Enter point 3 (x y)
Closest Pair: ((2, 7), (3, 6))
Minimum Distance: 1.4142135623730951
Time Complexity: O(n^2)
```

12. Write a program that finds the convex hull of a set of 2D points using the brute force approach.



```python
def orientation(p, q, r):
    return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])

def convex_hull(points):
    n = len(points)

    if n < 3:
        return points

    hull = []

    left = 0
    for i in range(1, n):
        if points[i][0] < points[left][0]:
            left = i

    p = left

    while True:
        hull.append(points[p])

        q = (p + 1) % n

        for i in range(n):
            if orientation(points[p], points[i], points[q]) < 0:
                q = i

        p = q

        if p == left:
            break

    return hull


points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
```

Output

Status : Successfully executed

Time: 0.0100 secs

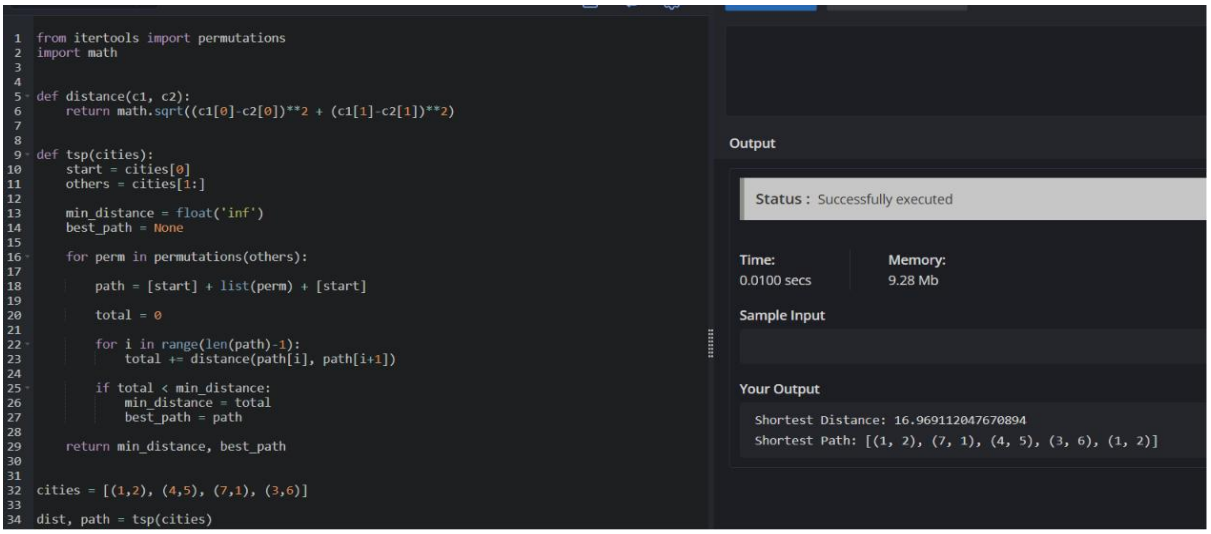
Memory: 9.1 Mb

Sample Input

Your Output

```
Convex Hull: [(0, 0), (8, 1), (4, 6)]
```

13. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP.



```python
from itertools import permutations
import math


def distance(c1, c2):
    return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)


def tsp(cities):
    start = cities[0]
    others = cities[1:]

    min_distance = float('inf')
    best_path = None

    for perm in permutations(others):

        path = [start] + list(perm) + [start]

        total = 0

        for i in range(len(path)-1):
            total += distance(path[i], path[i+1])

        if total < min_distance:
            min_distance = total
            best_path = path

    return min_distance, best_path


cities = [(1,2), (4,5), (7,1), (3,6)]

dist, path = tsp(cities)
```

Output

Status : Successfully executed

Time: 0.0100 secs

Memory: 9.28 Mb

Sample Input

Your Output

```
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
```

14. You are given a cost matrix where each element cost[i][j] represents the cost of assigning worker i to task j. Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function total_cost(assignment, cost_matrix) that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

```python
from itertools import permutations


def total_cost(assignment, cost_matrix):
    cost = 0

    for worker, task in enumerate(assignment):
        cost += cost_matrix[worker][task]

    return cost


def assignment_problem(cost_matrix):
    n = len(cost_matrix)

    min_cost = float('inf')
    best_assignment = None

    for assignment in permutations(range(n)):
        cost = total_cost(assignment, cost_matrix)

        if cost < min_cost:
            min_cost = cost
            best_assignment = assignment

    return best_assignment, min_cost


cost_matrix = [
    [3, 10, 7],
    [8, 5, 12],
    [4, 6, 9]
]

```

```
Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.256 Mb

Your Output

Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16
```

15. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:
    - Define a function total_value(items, values) that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
    - Define a function is_feasible(items, weights, capacity) that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

```python
from itertools import combinations


def total_value(items, values):
    return sum(values[i] for i in items)


def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity


def knapsack(weights, values, capacity):
    n = len(weights)

    best_value = 0
    best_items = []

    for r in range(n + 1):
        for subset in combinations(range(n), r):

            if is_feasible(subset, weights, capacity):

                value = total_value(subset, values)

                if value > best_value:
                    best_value = value
                    best_items = subset

    return best_items, best_value


weights = [2, 3, 1]
values = [4, 5, 3]
capacity = 4
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
9.272 Mb

Your Output

Optimal Selection: [1, 2]
Total Value: 8
```